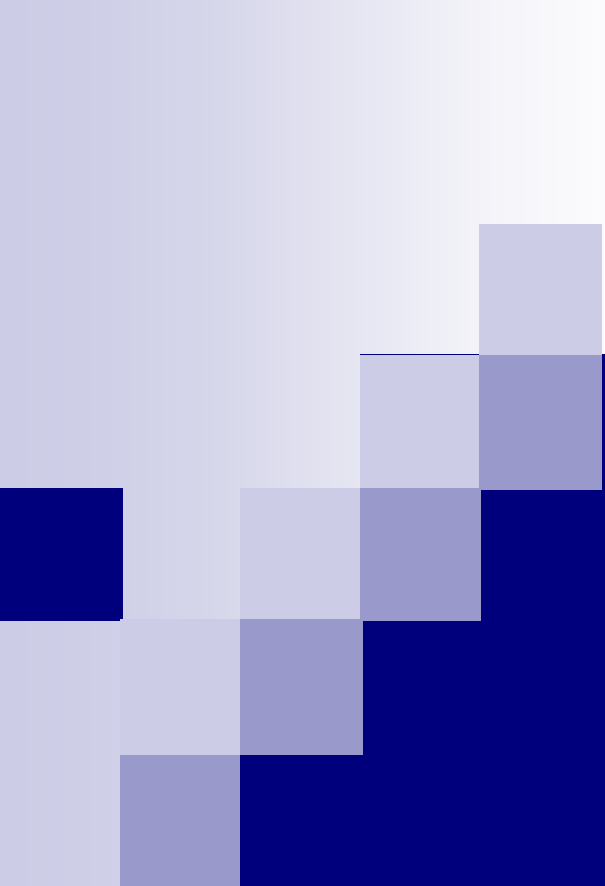




Stacks

Abstract Data Types (ADTs)

- An abstract data type (ADT) is an abstraction of a data structure
- An ADT specifies:
 - Data stored
 - Operations on the data
 - Error conditions associated with operations
- Example: ADT modeling a simple stock trading system
 - The data stored are buy/sell orders
 - The operations supported are
 - order **buy**(stock, shares, price)
 - order **sell**(stock, shares, price)
 - void **cancel**(order)
 - Error conditions:
 - Buy/sell a nonexistent stock
 - Cancel a nonexistent order

The Stack ADT

- The Stack ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Think of a spring-loaded plate dispenser
- Main stack operations:
 - **push**(object): inserts an element
 - object **pop**(): removes and returns the last inserted element
- Auxiliary stack operations:
 - object **top**(): returns the last inserted element without removing it
 - integer **size**(): returns the number of elements stored
 - boolean **isEmpty**(): indicates whether no elements are stored

Stack Interface in Java

- Java interface corresponding to our Stack ADT
- Requires the definition of class `EmptyStackException`
- Different from the built-in Java class `java.util.Stack`

```
public interface Stack {  
    public int size();  
    public boolean isEmpty();  
    public Object top()  
        throws EmptyStackException;  
    public void push(Object o);  
    public Object pop()  
        throws EmptyStackException;  
}
```

Exceptions

- Attempting the execution of an operation of ADT may sometimes cause an error condition, called an exception
- Exceptions are said to be “thrown” by an operation that cannot be executed
- In the Stack ADT, operations pop and top cannot be performed if the stack is empty
- Attempting the execution of pop or top on an empty stack throws an `EmptyStackException`



Applications of Stacks

■ Direct applications

- ☐ Page-visited history in a Web browser
- ☐ Undo sequence in a text editor
- ☐ Chain of method calls in the Java Virtual Machine

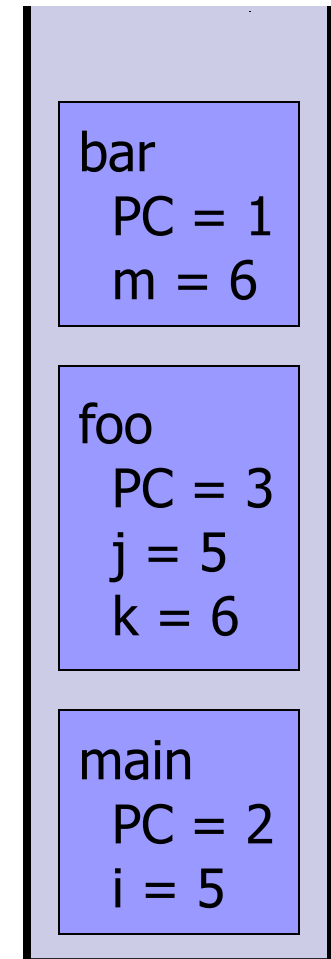
■ Indirect applications

- ☐ Auxiliary data structure for algorithms
- ☐ Component of other data structures

Method Stack in the JVM

- The Java Virtual Machine (JVM) keeps track of the chain of active methods with a stack
- When a method is called, the JVM pushes on the stack a frame containing
 - Local variables and return value
 - Program counter, keeping track of the statement being executed
- When a method ends, its frame is popped from the stack and control is passed to the method on top of the stack
- Allows for **recursion**

```
main() {  
    int i = 5;  
    foo(i);  
}  
  
foo(int j) {  
    int k;  
    k = j+1;  
    bar(k);  
}  
  
bar(int m) {  
    ...  
}
```

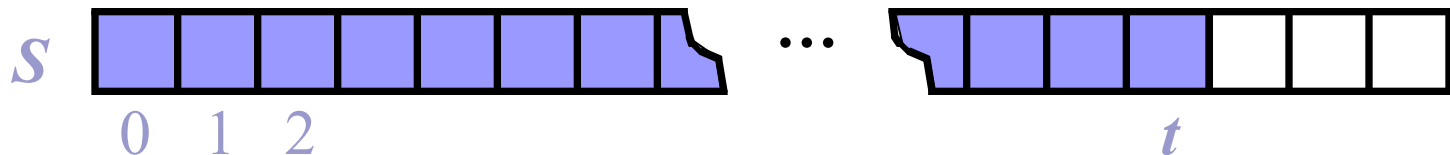


Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

```
Algorithm size()  
    return  $t + 1$ 
```

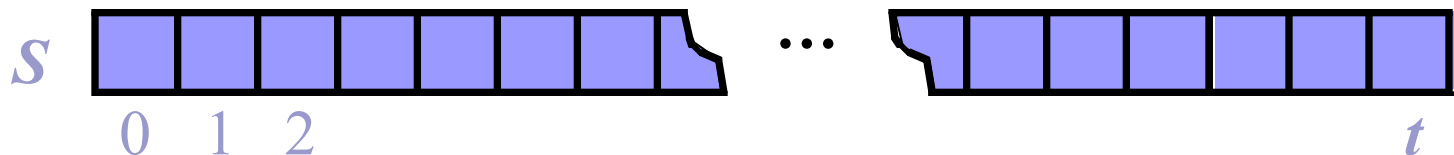
```
Algorithm pop()  
    if isEmpty() then  
        throw EmptyStackException  
    else  
         $t \leftarrow t - 1$   
        return  $S[t + 1]$ 
```



Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then throw a **FullStackException**
 - Limitation of the array-based implementation
 - Not intrinsic to the Stack ADT

```
Algorithm push(o)  
  if  $t = S.length - 1$  then  
    throw FullStackException  
  else  
     $t \leftarrow t + 1$   
     $S[t] \leftarrow o$ 
```



Using a Python list as a stack:

```
stack = []

# Push
stack.append('A')
stack.append('B')
stack.append('C')
print("Stack: ", stack)

# Peek
topElement = stack[-1]
print("Peek: ", topElement)

# Pop
poppedElement = stack.pop()
print("Pop: ", poppedElement)

# Stack after Pop
print("Stack after Pop: ", stack)

# isEmpty
isEmpty = not bool(stack)
print("isEmpty: ", isEmpty)

# Size
print("Size: ", len(stack))
```

Creating a stack using class:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, element):
        self.stack.append(element)

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack.pop()

    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack[-1]

    def isEmpty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

# Create a stack
myStack = Stack()

myStack.push('A')
myStack.push('B')
myStack.push('C')
```

Creating a Stack using a Linked List:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = None
        self.size = 0

    def push(self, value):
        new_node = Node(value)
        if self.head:
            new_node.next = self.head
        self.head = new_node
        self.size += 1

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        popped_node = self.head
        self.head = self.head.next
        self.size -= 1
        return popped_node.value

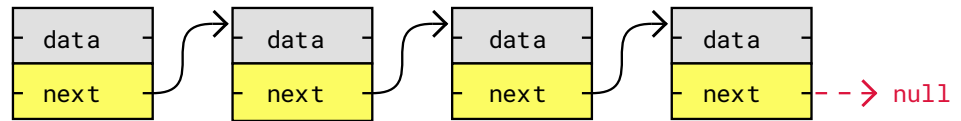
    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.head.value

    def isEmpty(self):
        return self.size == 0

    def stackSize(self):
        return self.size

    def traverseAndPrint(self):
        currentNode = self.head
        while currentNode:
            print(currentNode.value, end=" -> ")
            currentNode = currentNode.next
        print()
```

```
myStack = Stack()
myStack.push('A')
myStack.push('B')
myStack.push('C')
```



Performance and Limitations

■ Performance

- Let n be the number of elements in the stack
- The space used is $O(n)$
- Each operation runs in time $O(1)$

■ Limitations

- The maximum size of the stack must be defined a priori and cannot be changed
- Trying to push a new element into a full stack causes an implementation-specific exception

Array-based Stack in Java

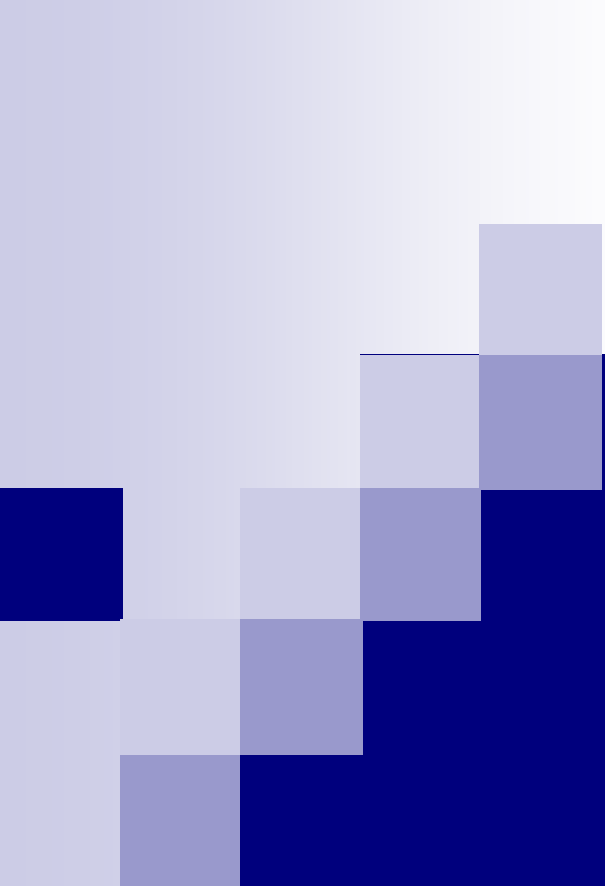
```
public class ArrayStack
    implements Stack {

    // holds the stack elements
    private Object S[ ];

    // index to top element
    private int top = -1;

    // constructor
    public ArrayStack(int
    capacity) {
        S = new
        Object[capacity];
    }
```

```
    public Object pop()
        throws EmptyStackException {
        if isEmpty()
            throw new
            EmptyStackException
            ("Empty stack: cannot pop");
        Object temp = S[top];
        // facilitates garbage collection
        S[top] = null;
        top = top - 1;
        return temp;
    }
```



Queues

The Queue ADT

- The Queue ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
 - enqueue(object): inserts an element at the end of the queue
 - object dequeue(): removes and returns the element at the front of the queue

Auxiliary queue operations:

- object front(): returns the element at the front without removing it
- integer size(): returns the number of elements stored
- boolean isEmpty(): indicates whether no elements are stored

Exceptions

- Attempting the execution of dequeue or front on an empty queue throws an `EmptyQueueException`

Queue Example

<i>Operation</i>		<i>Output</i> <i>Q</i>
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
dequeue()	5	(3)
enqueue(7)	—	(3, 7)
dequeue()	3	(7)
front()	7	(7)
dequeue()	7	()
dequeue()	"error"	()
isEmpty()	true	()
enqueue(9)	—	(9)
enqueue(7)	—	(9, 7)
size()	2	(9, 7)
enqueue(3)	—	(9, 7, 3)
enqueue(5)	—	(9, 7, 3, 5)
dequeue()	9	(7, 3, 5)



Applications of Queues

- Direct applications

- ☐ Waiting lists, bureaucracy
- ☐ Access to shared resources (e.g., printer)
- ☐ Multiprogramming

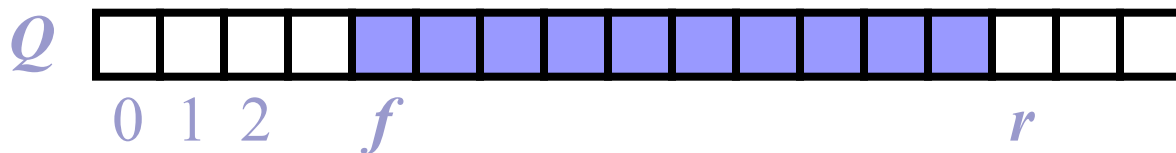
- Indirect applications

- ☐ Auxiliary data structure for algorithms
- ☐ Component of other data structures

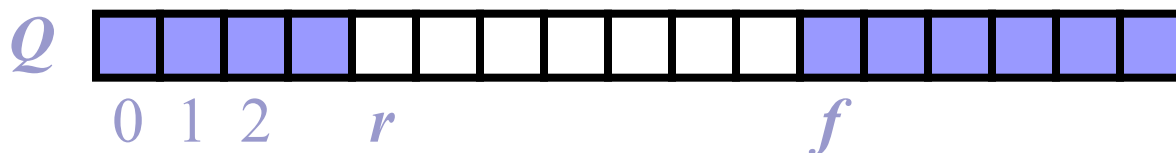
Array-based Queue

- Use an array of size N in a circular fashion
- Two variables keep track of the front and rear
 - f index of the front element
 - r index immediately past the rear element
- Array location r is kept empty

normal configuration



wrapped-around configuration

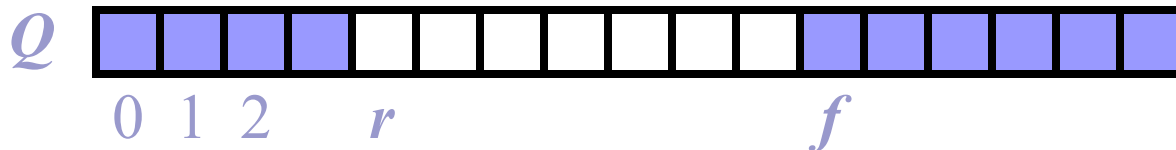
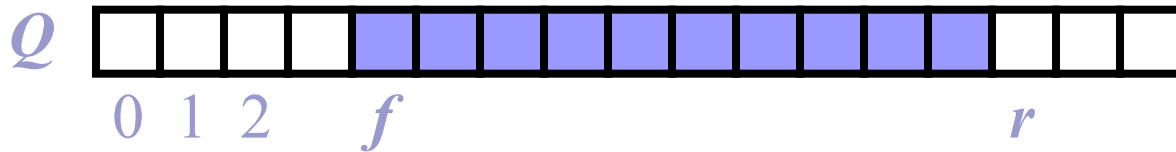


Queue Operations

- We use the modulo operator (remainder of division)

Algorithm *size()*
return $(N - f + r) \bmod N$

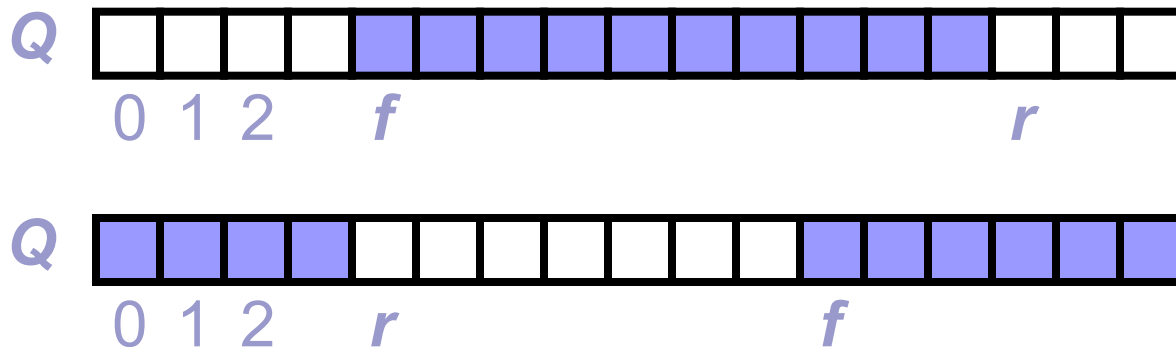
Algorithm *isEmpty()*
return $(f = r)$



Queue Operations (cont.)

- Operation enqueue throws an exception if the array is full
- This exception is implementation-dependent

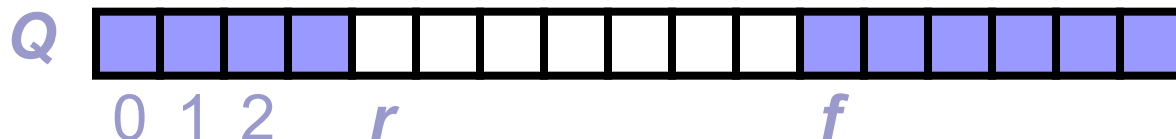
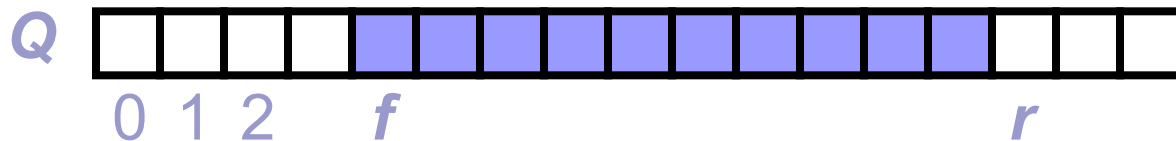
```
Algorithm enqueue(o)  
  if size() =  $N - 1$  then  
    throw FullQueueException  
  else  
     $Q[r] \leftarrow o$   
     $r \leftarrow (r + 1) \bmod N$ 
```



Queue Operations (cont.)

- Operation `dequeue` throws an exception if the queue is empty
- This exception is specified in the queue ADT

```
Algorithm dequeue()  
  if isEmpty() then  
    throw EmptyQueueException  
  else  
     $o \leftarrow Q[f]$   
     $f \leftarrow (f + 1) \bmod N$   
    return  $o$ 
```



Queue Interface in Java

- Java interface corresponding to our Queue ADT
- Requires the definition of class `EmptyQueueException`
- No corresponding built-in Java class

```
public interface Queue {  
    public int size();  
    public boolean isEmpty();  
    public Object front()  
        throws EmptyQueueException;  
    public void enqueue(Object o);  
    public Object dequeue()  
        throws  
EmptyQueueException;  
}
```

Using a Python list as a queue:

```
queue = []
```

```
# Enqueue
```

```
queue.append('A')  
queue.append('B')  
queue.append('C')  
print("Queue: ", queue)
```

```
# Peek
```

```
frontElement = queue[0]  
print("Peek: ", frontElement)
```

```
# Dequeue
```

```
poppedElement = queue.pop(0)  
print("Dequeue: ", poppedElement)
```

```
print("Queue after Dequeue: ", queue)
```

```
# isEmpty
```

```
isEmpty = not bool(queue)  
print("isEmpty: ", isEmpty)
```

```
# Size
```

```
print("Size: ", len(queue))
```

Using a Python class as a queue:

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, element):
        self.queue.append(element)

    def dequeue(self):
        if self.isEmpty():
            return "Queue is empty"
        return self.queue.pop(0)

    def peek(self):
        if self.isEmpty():
            return "Queue is empty"
        return self.queue[0]

    def isEmpty(self):
        return len(self.queue) == 0

    def size(self):
        return len(self.queue)

# Create a queue
myQueue = Queue()

myQueue.enqueue('A')
myQueue.enqueue('B')
myQueue.enqueue('C')
```

Creating a Queue using a Linked List:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.length = 0

    def enqueue(self, element):
        new_node = Node(element)
        if self.rear is None:
            self.front = self.rear = new_node
            self.length += 1
            return
        self.rear.next = new_node
        self.rear = new_node
        self.length += 1

    def dequeue(self):
        if self.isEmpty():
            return "Queue is empty"
        temp = self.front
        self.front = temp.next
        self.length -= 1
        if self.front is None:
            self.rear = None
        return temp.data

    def peek(self):
        if self.isEmpty():
            return "Queue is empty"
        return self.front.data

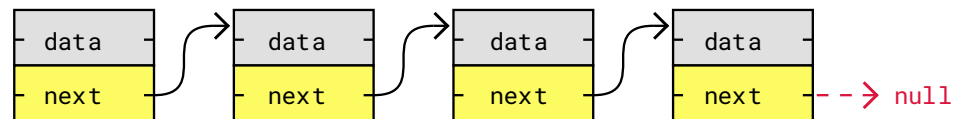
    def isEmpty(self):
        return self.length == 0

    def size(self):
        return self.length

    def printQueue(self):
        temp = self.front
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print()
```

Create a queue
myQueue = Queue()

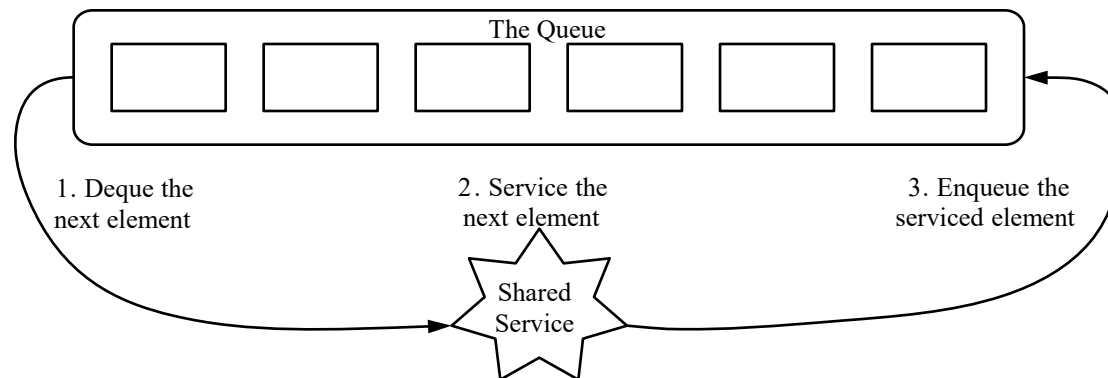
myQueue.enqueue('A')
myQueue.enqueue('B')
myQueue.enqueue('C')

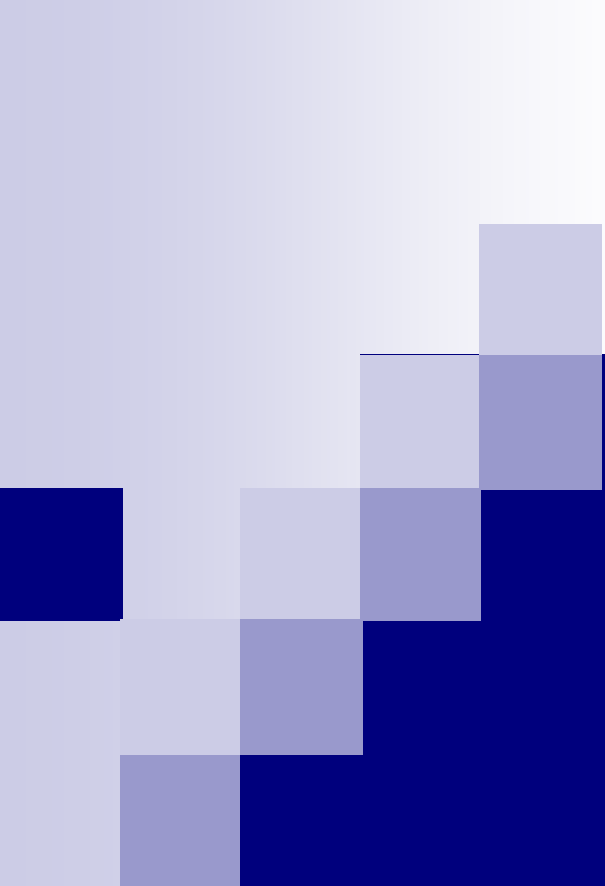


Application: Round Robin Schedulers

- We can implement a round robin scheduler using a queue, Q , by repeatedly performing the following steps:

1. $e = Q.dequeue()$
2. Service element e
3. $Q.enqueue(e)$

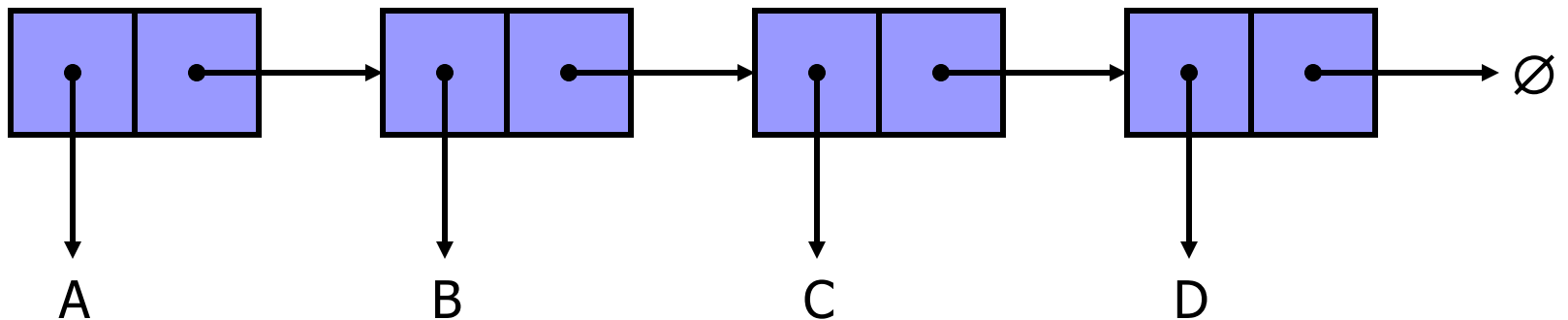
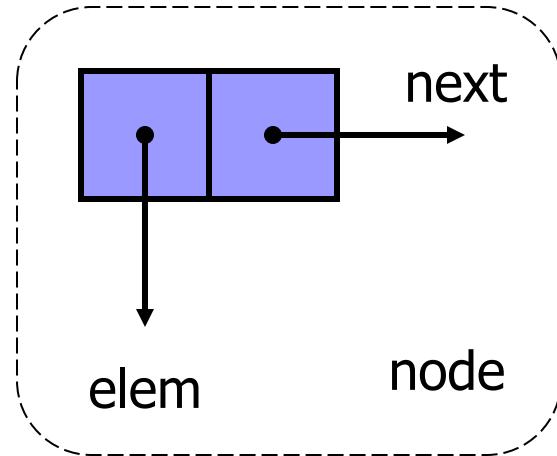




Linked Lists

Singly Linked List (§ 4.4.1)

- A singly linked list is a concrete data structure consisting of a sequence of nodes
- Each node stores
 - element
 - link to the next node

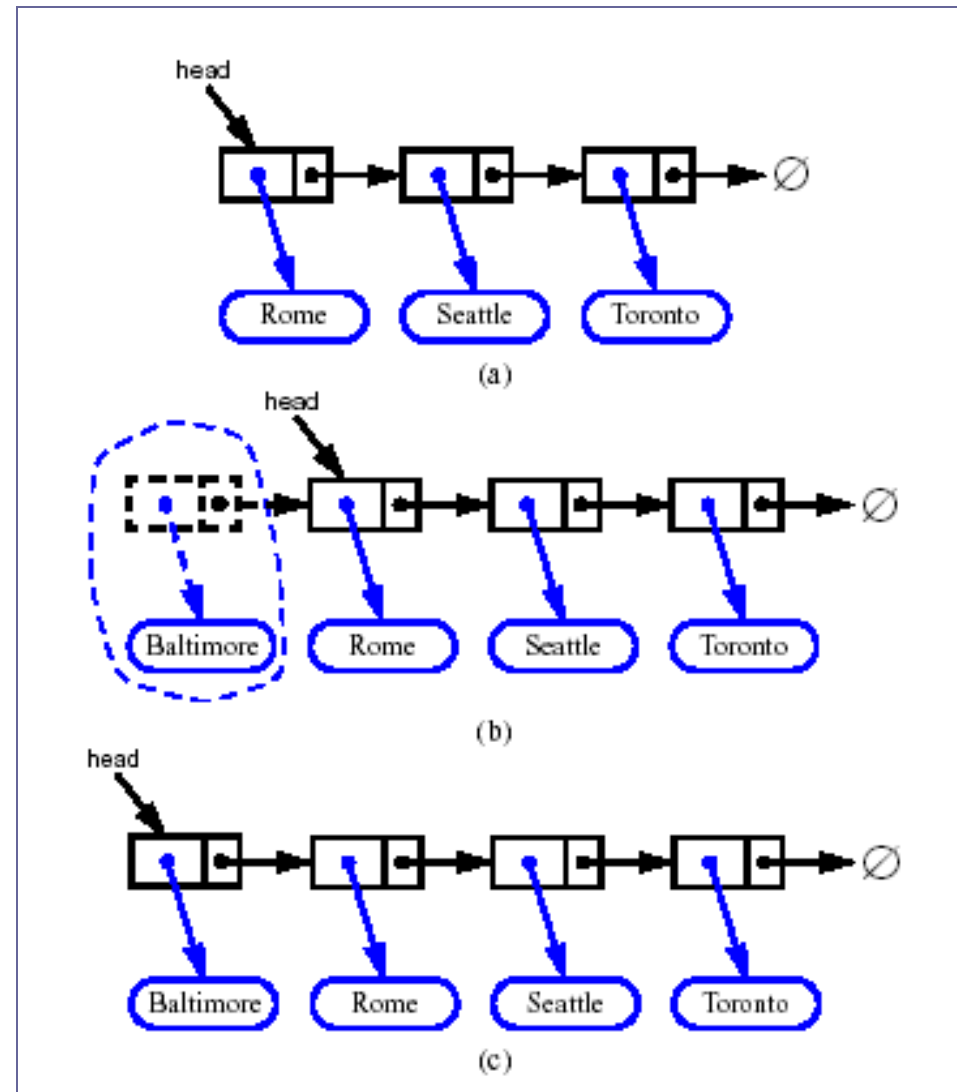


The Node Class for List Nodes

```
public class Node    {
    // Instance variables:
    private Object element;
    private Node next;
    /** Creates a node with null references to its element and next node. */
    public Node()    {
        this(null, null);
    }
    /** Creates a node with the given element and next node. */
    public Node(Object e, Node n) {
        element = e;
        next = n;
    }
    // Accessor methods:
    public Object getElement() {
        return element;
    }
    public Node getNext() {
        return next;
    }
    // Modifier methods:
    public void setElement(Object newElem) {
        element = newElem;
    }
    public void setNext(Node newNext) {
        next = newNext;
    }
}
```

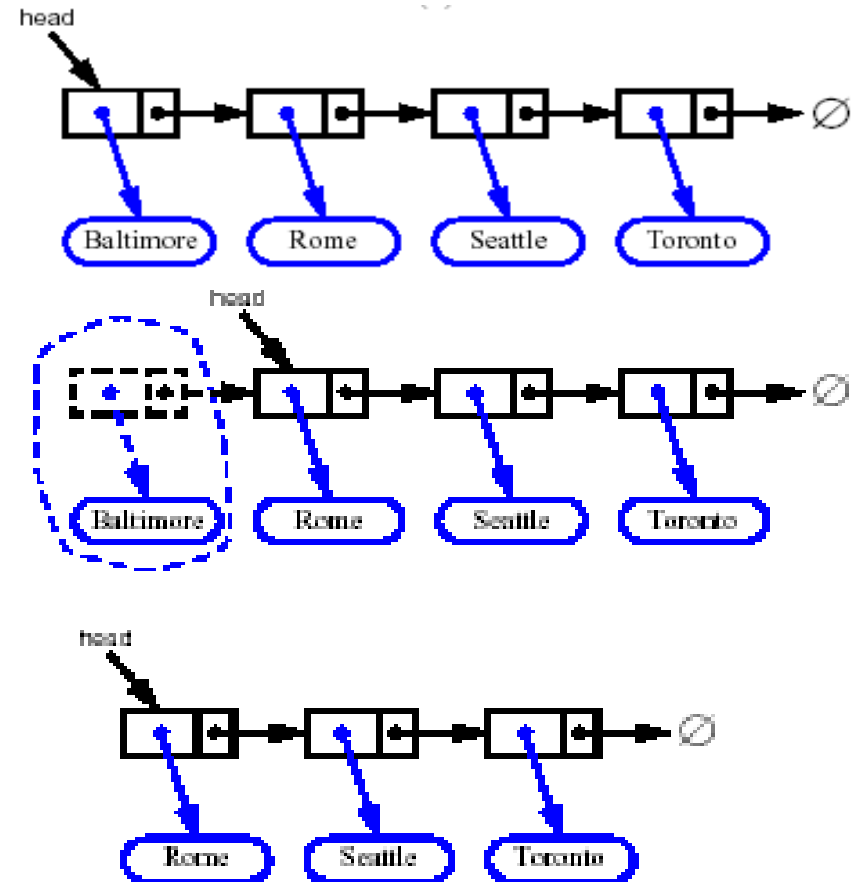
Inserting at the Head

1. Allocate a new node
2. Insert new element
3. Have new node point to old head
4. Update head to point to new node



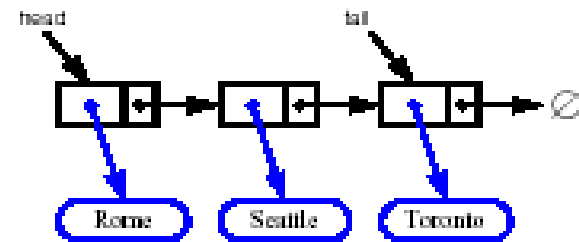
Removing at the Head

1. Update head to point to next node in the list
2. Allow garbage collector to reclaim the former first node

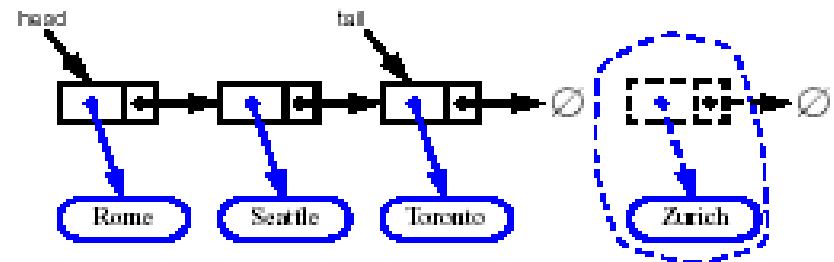


Inserting at the Tail

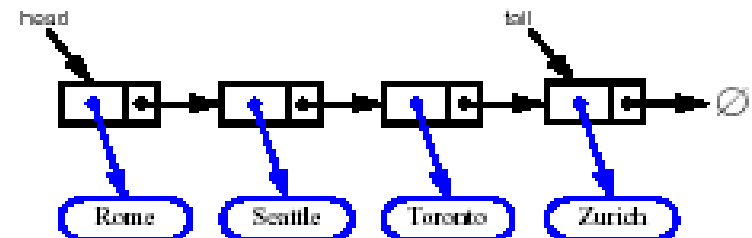
1. Allocate a new node
2. Insert new element
3. Have new node point to null
4. Have old last node point to new node
5. Update tail to point to new node



(a)



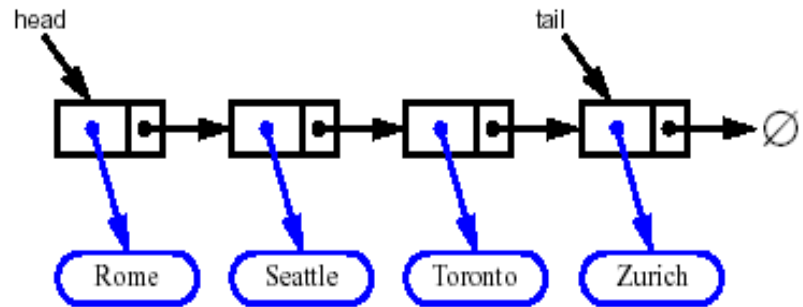
(b)



(c)

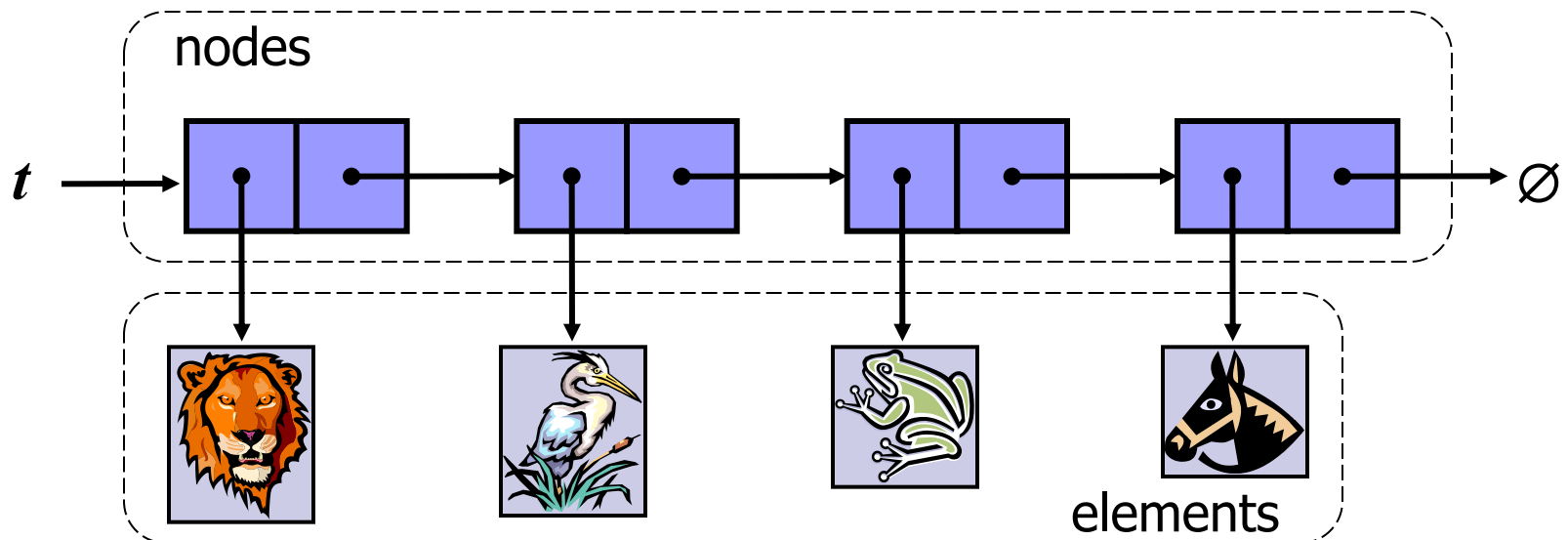
Removing at the Tail

- Removing at the tail of a singly linked list is not efficient!
- There is no constant-time way to update the tail to point to the previous node



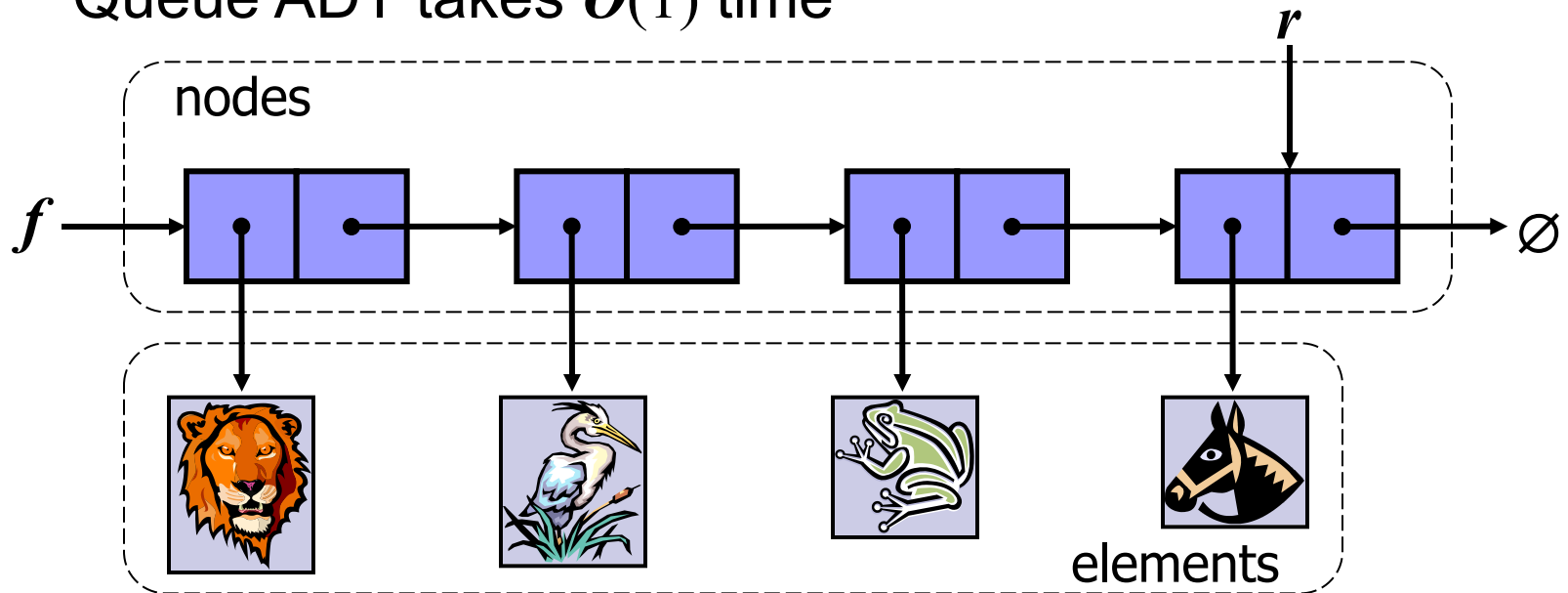
Stack with a Singly Linked List

- We can implement a stack with a singly linked list
- The top element is stored at the first node of the list
- The space used is $O(n)$ and each operation of the Stack ADT takes $O(1)$ time



Queue with a Singly Linked List

- We can implement a queue with a singly linked list
 - The front element is stored at the first node
 - The rear element is stored at the last node
- The space used is $O(n)$ and each operation of the Queue ADT takes $O(1)$ time



Double Ended Queues (Deque)

- The Deque ADT stores arbitrary objects
- Insertions and deletions can be done at both front and rear of the queue.
- Main queue operations:
 - insertFirst(e): Insert a new element *e* at beginning of the deque.
 - insertLast(e) Insert a new element *e* at the end of the deque.
 - object removeFirst()
 - object removeLast()
- Auxiliary queue operations:
 - object first():
 - object last():
 - int size():
 - boolean isEmpty():
- Exceptions
 - Attempting the execution of dequeue or front on an empty double ended queue throws an `EmptyQueueException`

Deque with Doubly Linked List

- Since the deque requires insertion and removal at both ends of a list, therefore implement with doubly linked list is a good solution.

Method	Time
size(), isEmpty()	
first, last	
insertFirst, insertLast	
removeFirst, removeLast	



Acknowledgement

- Data Structures and Algorithm in JAVA:
Michael T. Goodrich and Roberto Tamassia
- www.datastructures.net
- <http://www.site.uottawa.ca/~rlantos/CSI2114/PDFslides/04-stackSpam.pdf>